

Roteiro tecnico para apresentacao - Portal de Notas com Kerberos

Este roteiro foi feito para ser lido durante a apresentacao. Ele esta um pouco mais tecnico do que a versao anterior, porque o objetivo e deixar evidente que o projeto atende aos requisitos do trabalho.

Como usar:

- O texto entre parenteses indica o que mostrar na tela.
- A frase depois de "Frase para ler:" e o que deve ser falado.
- As linhas indicadas sao as linhas atuais do projeto nesta versao.
- Se alguma linha mudar depois, procure pelo nome da funcao indicado.
- Nao leia as instrucoes entre parenteses; elas servem como guia de tela.

Tempo sugerido: 18 a 24 minutos.

Divisao:

- Apresentador 1: arquitetura Kerberos, criptografia simetrica, senha, KDF, AS e TGT.
- Apresentador 2: TGS, Service Ticket, autenticadores, replay, autenticacao mutua e operacoes protegidas.
- Apresentador 3: Portal de Notas, perfis, interface, logs e demonstracao professor/aluno.
- Apresentador 4: execucao, testes, Doxygen, limitacoes e conclusao.

Mensagem principal:

O projeto implementa o fluxo Kerberos academico completo: Cliente, AS, TGS e Servico. A senha e usada apenas para derivar uma chave no cliente. O AS emite o TGT, o TGS emite o Service Ticket e o Portal de Notas valida ticket, autenticador, timestamp, nonce e hash antes de executar as operacoes.

Mapa rapido dos requisitos e onde mostrar

Use esta tabela como cola tecnica. Ela tambem ajuda se o professor perguntar onde cada requisito aparece no codigo.

Requisito	Onde mostrar	O que comentar
Kerberos com criptografia simetrica	crypto_utils.py linhas 25, 132, 167, 184 e 218; config.py linhas 38 a 48	O projeto usa AES-GCM e chaves simetricas compartilhadas, sem biblioteca pronta de Kerberos.
Servidor AS	servidor_as.py linhas 19, 38, 47 e 48; as_server.py linhas 128 e 276	O AS entrega desafio, valida prova HMAC e emite resposta com TGT.
Servidor TGS	servidor_tgs.py linhas 16, 33 e 41; tgs_server.py linhas 99, 152 e 224	O TGS valida TGT, valida autenticador Cliente-TGS e emite Service Ticket.
Servico protegido por Kerberos	servidor_notas.py linhas 19, 34 a 46; portal_notas.py linhas 38, 192 e 401	O servico protegido e o Portal de Notas, acessado com Service Ticket e autenticador.
Autenticacao por senha	routes.py linhas 719 e 720; kdf.py linhas 69 e 106 a 111	O usuario faz login com senha, e o cliente deriva a chave localmente.
KDF	kdf.py linhas 29 a 31, 69 e 106 a 111	PBKDF2-HMAC-SHA256, salt de 16 bytes, chave de 32 bytes e 200.000 iteracoes.
Emissao e validacao de tickets	as_server.py linhas 246 a 254; tickets.py linhas 46 e 72; tgs_server.py linhas	

256 a 270; portal_notas.py linhas 60 a 72 | AS emite TGT, TGS emite Service Ticket e Portal valida Service Ticket. |
Autenticacao mutua	portal_notas.py linhas 222 a 224; routes.py linha 420; portal_notas.py linhas 240, 287 e 295	Portal responde cifrado com timestamp + 1 e nonce; o cliente valida.
Fluxo Cliente, AS, TGS e Servico	routes.py linhas 87, 130, 159, 182, 208, 270, 291, 396, 420 e 518	O Cliente Web coordena o fluxo completo e depois protege cada operacao.
Operacoes protegidas no Portal	routes.py linhas 537, 560 a 570, 603 e 637; portal_notas.py linhas 401, 482, 514 e 530	Cada acao usa requisicao cifrada, hash, nonce, autenticador e resposta validada.
Professor e aluno	service.py linhas 22, 23, 53, 71, 101, 143, 186 e 222	Professor altera notas; aluno apenas consulta.
Testes automatizados	test_rede.py linhas 96, 191 e 245; test_notas.py linhas 170, 203, 278 e 338	Os testes comprovam sockets TCP, senha fora da rede, autenticacao mutua, replay e autorizacao.

Preparacao antes de gravar

(Mostre dois terminais na raiz do projeto.)

Frase para ler:

Antes de comecar, eu deixo dois terminais abertos. Um terminal fica para os servidores Kerberos e outro para o Flask, que e o Cliente Web.

(No primeiro terminal, mostre o comando.)

```
python scripts/iniciar_servidores.py
```

Frase para ler:

Neste terminal, o script sobe os tres servidores do fluxo Kerberos: AS, TGS e Portal de Notas.

(No segundo terminal, mostre o comando.)

```
python run.py
```

Frase para ler:

Neste outro terminal, o Flask fica rodando na porta 5000. Ele e a parte web, mas por baixo conversa com os servidores Kerberos via TCP.

(Mostre o navegador em http://127.0.0.1:5000.)

Frase para ler:

Com isso, a aplicacao fica acessivel em http://127.0.0.1:5000.

Apresentador 1 - Arquitetura, criptografia, senha, KDF, AS e TGT

1. Abertura e objetivo

(Mostre README.md, titulo e arquitetura.)

Frase para ler:

O nosso projeto e um Portal de Notas Escolares protegido por uma simulacao academica do protocolo Kerberos. O objetivo nao e usar uma biblioteca pronta, mas implementar o fluxo principal do Kerberos com primitivas criptograficas basicas.

(Mostre ainda o README.md, na parte em que aparece o fluxo ou a arquitetura.)

Frase para ler:

A ideia e que o usuario nao acesse o servico apenas com usuario e senha. Ele passa primeiro pelo AS, depois pelo TGS, e so entao acessa o Portal de Notas com um ticket de servico.

2. Separacao real entre Cliente, AS, TGS e Servico

(Mostre config.py linhas 55 a 58.)

Frase para ler:

Aqui no config.py, linhas 55 a 58, aparecem o host e as portas dos servidores: AS na 9001, TGS na 9002 e Portal de Notas na 9003. Essa separacao ajuda a demonstrar o fluxo Cliente, AS, TGS e Servico.

(Mostre run.py linha 22.)

Frase para ler:

No run.py, linha 22, o Flask e criado com usar_rede=True. Isso significa que a execucao normal usa os servidores TCP reais, e nao apenas chamadas locais de funcao.

(Mostre scripts/iniciar_servidores.py linhas 57 a 66.)

Frase para ler:

Em scripts/iniciar_servidores.py, linhas 57 a 66, vemos os tres processos: um para o AS, um para o TGS e um para o Portal de Notas. Isso mostra que os componentes do Kerberos rodam separados.

(Mostre rede/servidor.py linhas 16 e 86.)

Frase para ler:

A camada de rede usa um servidor TCP compartilhado. Em rede/servidor.py, linha 16, aparece o ThreadingTCPServer, e na linha 86 a funcao que cria o servidor TCP para cada componente.

(Mostre rede/protocolo.py linhas 42 e 62.)

Frase para ler:

Em rede/protocolo.py, linhas 42 e 62, ficam as funcoes de enviar e receber mensagens JSON pelo socket. Entao a troca Cliente, AS, TGS e Portal realmente acontece por rede local.

(Mostre rede/cliente_tcp.py linhas 134, 141, 153, 172 e 183.)

Frase para ler:

E em cliente_tcp.py, as linhas 134, 141, 153, 172 e 183 mostram as chamadas do Cliente para o AS, para o TGS e para o Portal. Isso fecha a evidencia da separacao do fluxo.

3. Criptografia simetrica

(Mostre config.py linhas 38 a 48.)

Frase para ler:

O requisito pede Kerberos usando criptografia de chave simetrica. No config.py, linhas 38 a 48, o projeto carrega a chave secreta do TGS e a chave secreta do servico de notas. Essas chaves sao compartilhadas com os servidores correspondentes.

(Mostre crypto_utils.py linha 25.)

Frase para ler:

Em crypto_utils.py, linha 25, aparece o uso de AES-GCM. Esse e o algoritmo usado para cifrar e autenticar os pacotes JSON.

(Mostre crypto_utils.py linhas 132 e 167.)

Frase para ler:

A funcao criptografar_json, linha 132, serializa um dicionario e cifra com AES-GCM. Na linha 167, o objeto AESGCM e criado com a chave simetrica.

(Mostre crypto_utils.py linhas 184 e 218.)

Frase para ler:

A funcao descriptografar_json, linha 184, faz o caminho contrario. Na linha 218, o AES-GCM tenta abrir o pacote. Se a chave estiver errada ou se o dado tiver sido adulterado, a abertura falha.

4. Senha e derivacao de chave

(Mostre a tela de login ou routes.py linhas 719 e 720.)

Frase para ler:

A autenticao do usuario comeca na rota /login. Em routes.py, linhas 719 e 720, fica a rota de login. O usuario informa usuario e senha na interface.

(Mostre kdf.py linhas 29 a 31.)

Frase para ler:

A senha nao e enviada diretamente para os servidores. Em kdf.py, linhas 29 a 31, temos os parametros da KDF: salt de 16 bytes, chave de 32 bytes e 200.000 iteracoes.

(Mostre kdf.py linhas 69 e 106 a 111.)

Frase para ler:

A funcao derivar_chave_senha, linha 69, usa PBKDF2-HMAC-SHA256. Nas linhas 106 a 111, vemos a chamada hashlib.pbkdf2_hmac, usando SHA-256, senha, salt, numero de iteracoes e tamanho final da chave.

(Mostre kdf.py linhas 188 e 227.)

Frase para ler:

Depois da derivacao, o cliente cria uma prova HMAC. A funcao gerar_prova_as, linha 188, monta essa prova, e na linha 227 aparece o hmac.new. Assim, o cliente prova que conhece a senha sem enviar a senha.

(Mostre data/usuarios.json, apenas os campos salt, verificador e perfil.)

Frase para ler:

No arquivo de usuarios, nao existe senha em texto claro. O arquivo guarda salt, verificador e perfil. Isso reforca que a senha e usada apenas para reproduzir a chave no momento do login.

5. AS e desafio

(Mostre servidor_as.py linhas 19, 38, 47 e 48.)

Frase para ler:

O AS fica exposto por TCP em servidor_as.py. A linha 19 mostra o processador de requisicoes do AS. A linha 38 trata a acao de obter parametros, e as linhas 47 e 48 tratam a autenticao com a prova HMAC.

(Mostre as_server.py linhas 47 e 48.)

Frase para ler:

Em as_server.py, linhas 47 e 48, existe o armazenamento em memoria dos desafios do AS e o lock usado para proteger esse acesso.

(Mostre as_server.py linhas 128, 179, 180 e 187.)

Frase para ler:

A funcao criar_desafio_as, linha 128, cria os parametros iniciais. A linha 179 gera um desafio aleatorio, a linha 180 registra esse desafio em memoria e a linha 187 devolve as iteracoes da KDF para o cliente.

(Mostre routes.py linhas 130, 159, 182 e 208.)

Frase para ler:

Do lado do cliente, em routes.py, linha 130, o Cliente pede os parametros ao AS. Na linha 159 ele deriva a chave da senha, na linha 182 gera a prova HMAC e na linha 208 envia essa prova ao AS.

(Mostre as_server.py linhas 276, 330, 353 e 358.)

Frase para ler:

A validacao fica em autenticar_no_as_com_prova, linha 276. Na linha 330 o desafio e consumido com pop, na linha 353 o AS calcula a prova esperada, e na linha 358 compara com a prova recebida usando hmac.compare_digest.

6. Emissao do TGT

(Mostre as_server.py linhas 198, 246, 247 e 254.)

Frase para ler:

Quando a prova esta correta, o AS emite a resposta. Em as_server.py, linha 198, comeca _emitir_resposta_as. Nas linhas 246 e 247, o AS gera a chave de sessao Cliente-TGS. Na linha 254, o TGT e cifrado com a chave secreta do TGS.

(Mostre tickets.py linhas 46 e 66.)

Frase para ler:

A estrutura do TGT esta em tickets.py, linha 46. Na linha 66 aparece o campo chave_sessao_cliente_tgs, que e a chave de sessao entregue ao TGS dentro do ticket.

(Mostre as_server.py linhas 257 e 382.)

Frase para ler:

Na resposta para o cliente, a linha 257 inclui a chave Cliente-TGS. A funcao gerar_tgt, linha 382, monta os campos adicionais de validade, usuario e nonce do TGT.

Fechamento do Apresentador 1:

Resumindo: ate aqui cobrimos autenticacao por senha, KDF, desafio do AS, prova HMAC, emissao do TGT e uso de criptografia simetrica. Agora o Cliente usa esse TGT para pedir ao TGS um ticket especifico para o Portal de Notas.

Apresentador 2 - TGS, Service Ticket, autenticadores e autenticacao mutua

1. Entrada no TGS

(Mostre servidor_tgs.py linhas 16, 33 e 41.)

Frase para ler:

O TGS fica em servidor_tgs.py. A linha 16 mostra o processador de requisicao. A linha 33 exige a acao emitir_ticket, e na linha 41 o servidor chama a funcao que emite o Service Ticket.

(Mostre tgs_server.py linhas 38, 99, 152 e 224.)

Frase para ler:

A logica principal fica em tgs_server.py. A linha 38 mostra que o servico cadastrado e notas. A linha 99 valida o TGT, a linha 152 valida o autenticador Cliente-TGS e a linha 224 inicia a emissao do ticket de servico.

2. Autenticador Cliente-TGS

(Mostre authenticator.py linhas 21 a 26.)

Frase para ler:

O autenticador e criado em authenticator.py, a partir da linha 21. Ele recebe usuario, chave de sessao, nonce, acao e hash da requisicao quando esses campos sao necessarios.

(Mostre authenticator.py linhas 42, 43, 47 e 50.)

Frase para ler:

Nas linhas 42 e 43, o autenticador recebe timestamp e nonce. Nas linhas 47 e 50, ele tambem pode receber a acao e o hash da requisicao. Isso e usado nas operacoes protegidas do Portal.

(Mostre routes.py linhas 270 e 291.)

Frase para ler:

No Cliente Web, a linha 270 cria o autenticador Cliente-TGS. Depois, na linha 291, o Cliente pede ao TGS o ticket de servico para o Portal de Notas.

3. Validacao do TGT e protecao contra replay

(Mostre tgs_server.py linhas 114 e 136.)

Frase para ler:

Na validacao do TGT, a linha 114 abre o ticket com a chave secreta do TGS. A linha 136 verifica se o ticket expirou. Isso mostra a validacao do ticket emitido pelo AS.

(Mostre tgs_server.py linhas 194, 199, 207 e 215.)

Frase para ler:

Na validacao do autenticador, a linha 194 pega o timestamp. A linha 199 rejeita autenticador expirado, a linha 207 rejeita timestamp futuro invalido e a linha 215 registra o nonce para proteger contra replay.

(Mostre tgs_server.py linhas 35, 59, 83 e 91.)

Frase para ler:

A protecao contra replay aparece no cache NONCES_TGS_UTILIZADOS, linha 35. A funcao _registrar_nonce_tgs, linha 59, verifica se o nonce ja apareceu. Na linha 83, se o par usuario e nonce ja existe, o TGS rejeita. Na linha 91, um nonce valido e registrado.

4. Emissao do Service Ticket

(Mostre tgs_server.py linhas 256, 267 e 270.)

Frase para ler:

Depois que o TGT e o autenticador sao validados, o TGS cria o Service Ticket. A linha 256 chama criar_ticket_servico. A linha 267 cifra esse ticket com a chave do servico, e a linha 270 monta a resposta cifrada para o cliente.

(Mostre tickets.py linhas 72 a 90.)

Frase para ler:

A estrutura do Service Ticket fica em tickets.py, a partir da linha 72. Ele contém usuário, serviço, chave Cliente-Serviço, timestamp de emissão, expiração e nonce.

(Mostre tgs_server.py linhas 284 e 285.)

Frase para ler:

Na resposta final do TGS, a linha 284 entrega o Service Ticket cifrado, e a linha 285 entrega a resposta para o cliente com a chave Cliente-Serviço protegida.

5. Portal de Notas como serviço protegido

(Mostre servidor_notas.py linhas 19, 34 a 46.)

Frase para ler:

O serviço protegido é o Portal de Notas. Em servidor_notas.py, linha 19, está o processador do serviço. As linhas 34 a 46 mostram as duas ações aceitas: autenticar no Portal e executar uma operação protegida.

(Mostre portal_notas.py linhas 38, 60 e 72.)

Frase para ler:

Em portal_notas.py, linha 38, o serviço é identificado como notas. A função validar_ticket_portal, linha 60, valida o Service Ticket, e a linha 72 chama a abertura do ticket para esse serviço.

(Mostre tgs_server.py linhas 295, 312 e 332.)

Frase para ler:

A abertura real do Service Ticket está em tgs_server.py, linha 295. Na linha 312, o ticket é descriptografado com a chave do serviço. Na linha 332, o sistema verifica se o ticket expirou.

6. Autenticação mútua

(Mostre portal_notas.py linhas 192, 222, 223 e 224.)

Frase para ler:

A autenticação inicial com o Portal fica em autenticar_portal_notas, linha

192. Na resposta, a linha 222 coloca o serviço, a linha 223 devolve o timestamp incrementado em 1, e a linha 224 devolve o mesmo nonce do autenticador.

(Mostre routes.py linhas 396, 420 e 456.)

Frase para ler:

Do lado do Cliente, a linha 396 envia o Service Ticket e o autenticador ao Portal. A linha 420 valida a confirmação do Portal. Depois, a linha 456 marca a sessão como portal_autenticado.

(Mostre portal_notas.py linhas 240, 287 e 295.)

Frase para ler:

A validação da confirmação fica em validar_confirmacao_portal, linha 240. A linha 287 confere o timestamp incrementado, e a linha 295 confere se o nonce recebido é o mesmo nonce esperado.

7. Operações protegidas depois do login

(Mostre routes.py linha 518.)

Frase para ler:

O ponto mais importante para reforçar é que o Kerberos não fica só no login. A função executar_operacao_kerberos, linha 518, protege cada operação feita no Portal de Notas.

(Mostre routes.py linhas 537, 560, 561 e 570.)

Frase para ler:

A linha 537 monta a requisição com usuário, ação, dados e nonce. A linha 560 calcula o hash da requisição. A linha 561 cifra a requisição com AES-GCM, e a linha 570 coloca o hash no autenticador.

(Mostre routes.py linhas 603 e 637.)

Frase para ler:

A linha 603 envia a operação ao Portal via TCP. Depois, na linha 637, o Cliente valida a resposta cifrada do Portal antes de aceitar o resultado.

(Mostre portal_notas.py linhas 401, 482, 514 e 530.)

Frase para ler:

No Portal, a função processar_operacao_portal, linha 401, valida a operação.

A linha 482 compara o hash com `hmac.compare_digest`. A linha 514 monta a resposta com timestamp incrementado, e a linha 530 inicia a validacao da resposta do lado do Cliente.

Fechamento do Apresentador 2:

Entao esta parte comprova TGS, Service Ticket, autenticadores, timestamp, nonce, replay, autenticacao mutua e operacoes protegidas. Isso mostra que o fluxo segue Cliente, AS, TGS e Servico, como no Kerberos apresentado em sala.

Apresentador 3 - Portal de Notas, perfis, interface e logs

1. Demonstracao como professor

(Mostre o navegador em `http://127.0.0.1:5000`.)

Frase para ler:

Agora vamos ver o servico protegido funcionando. O Portal de Notas e a parte visivel da aplicacao, mas o acesso a ele depende do fluxo Kerberos que acabou de ser explicado.

(Mostre a tela de login e entre com `SilvioSants`, sem mostrar a senha.)

Frase para ler:

Vou entrar com um usuario professor. A senha nao sera mostrada, mas ela sera usada pelo Cliente para derivar a chave localmente e iniciar o fluxo com o AS.

(Mostre o painel de professor.)

Frase para ler:

Depois do login, o painel de professor aparece. Isso significa que o Cliente ja passou por AS, TGS e Portal, recebeu o Service Ticket e concluiu a autenticacao mutua.

(Mostre o formulario de lancamento de nota.)

Frase para ler:

Como este usuario tem perfil de professor, aparecem os controles para lancar nota. Vou cadastrar uma nota para um aluno, por exemplo `AkinGOD777`.

(Preencha uma nota simples e clique para salvar.)

Frase para ler:

Ao salvar, a operacao nao vai direto para o JSON. Ela passa por executar_operacao_kerberos: a requisicao e cifrada, recebe nonce, hash e autenticador, e so depois o Portal executa a regra de negocio.

(Mostre a nota aparecendo na tabela.)

Frase para ler:

A nota aparece na tabela depois que o Portal valida a operacao e devolve uma resposta cifrada que o Cliente tambem valida.

2. Rotas Flask do Portal

(Mostre routes.py linhas 750, 847, 890 e 923.)

Frase para ler:

As rotas principais do Portal aparecem em routes.py. A linha 750 trata a rota /notas, a linha 847 trata a edicao, a linha 890 trata a exclusao e a linha 923 trata o logout.

(Mostre routes.py linhas 750 a 846, se quiser comentar de forma geral.)

Frase para ler:

A rota /notas lista o painel e tambem recebe o formulario de lancamento de nota. Quando o professor envia uma nota, essa rota chama a operacao protegida pelo Kerberos.

3. Regras de professor e aluno

(Mostre service.py linhas 22 e 23.)

Frase para ler:

As regras de perfil ficam em service.py. As linhas 22 e 23 definem os dois perfis principais: professor e aluno.

(Mostre service.py linhas 53, 65 e 66.)

Frase para ler:

A funcao listar_notas, linha 53, diferencia os perfis. Se for professor, a linha 65 permite listar todas as notas. Se for aluno, ele recebe apenas as notas do proprio usuario.

(Mostre service.py linhas 71, 78 e 79.)

Frase para ler:

A funcao `_validar_professor`, linha 71, bloqueia operacoes de alteracao para quem nao e professor. Nas linhas 78 e 79, se o perfil nao for professor, o sistema levanta erro de permissao.

(Mostre `service.py` linhas 101, 143, 186 e 222.)

Frase para ler:

As funcoes de criar nota, criar varias notas, editar e excluir ficam nas linhas 101, 143, 186 e 222. Todas dependem da validacao de perfil para alterar dados.

4. Demonstracao como aluno

(Faca logout e entre com `AkinGOD777`, sem mostrar a senha.)

Frase para ler:

Agora vamos entrar com um usuario aluno. Ele tambem passa pelo Kerberos, mas recebe permissoes diferentes dentro do Portal.

(Mostre o painel do aluno.)

Frase para ler:

No painel do aluno, ele ve apenas as proprias notas. Os controles de lancar, editar e excluir nao aparecem.

(Mostre a ausencia dos botoes e do formulario.)

Frase para ler:

A interface ja evita que o aluno use operacoes de professor, mas a regra mais importante esta no servidor, em `service.py`. Mesmo que alguem tente chamar uma rota diretamente, a camada de servico bloqueia.

5. Persistencia em JSON

(Mostre `repository.py` linhas 79, 128 e 154.)

Frase para ler:

A persistencia das notas fica em `repository.py`. A linha 79 adiciona nota, a linha 128 atualiza nota por ID e a linha 154 exclui nota por ID.

(Mostre `json_store.py` linhas 20, 36 e 58.)

Frase para ler:

A leitura e escrita de JSON ficam em `json_store.py`. A linha 20 carrega JSON,

a linha 36 salva JSON e a linha 58 usa os.replace, que substitui o arquivo final depois de gravar um temporario.

6. Logs didaticos

(Mostre o terminal dos servidores Kerberos.)

Frase para ler:

Os logs do terminal mostram o fluxo em tempo real. E possivel acompanhar o AS recebendo requisicao, o TGS emitindo ticket e o Portal validando operacao.

(Mostre logs.py linhas 13, 88, 142, 151, 158, 169 e 173.)

Frase para ler:

O arquivo logs.py centraliza os logs. A lista de campos sensiveis comeca na linha 13. A funcao de mascaramento esta na linha 88. As funcoes de log para interface e terminal aparecem nas linhas 142, 151, 158, 169 e 173.

Frase para ler:

Isso ajuda na apresentacao porque mostra o fluxo, mas sem expor senha, chaves, tickets, autenticadores, nonces ou ciphertxts.

Fechamento do Apresentador 3:

Nesta parte, vimos que existe um sistema de notas real por tras do Kerberos: professor cria e altera notas, aluno apenas consulta, as permissoes sao aplicadas no servidor e os logs ajudam a demonstrar o fluxo.

Apresentador 4 - Execucao, testes, Doxygen, limitacoes e conclusao

1. Execucao reproduzivel

(Mostre README.md, secao de instalacao e execucao.)

Frase para ler:

A execucao do projeto esta documentada no README. A instalacao pode ser feita com python -m pip install -e ..

(Mostre o terminal dos servidores.)

Frase para ler:

Para rodar, primeiro iniciamos AS, TGS e Portal com

python scripts/iniciar_servidores.py.

(Mostre o terminal Flask.)

Frase para ler:

Depois iniciamos o Cliente Web com python run.py e acessamos

http://127.0.0.1:5000.

2. Testes automatizados

(Mostre a pasta tests.)

Frase para ler:

O projeto possui testes automatizados para comprovar as partes principais do fluxo.

(Mostre test_crypto.py linha 38.)

Frase para ler:

Em test_crypto.py, linha 38, existe teste que detecta ciphertext adulterado.

Isso comprova a integridade fornecida pelo AES-GCM.

(Mostre test_as_server.py linhas 92 e 105.)

Frase para ler:

Em test_as_server.py, a linha 92 testa autenticacao valida no AS, e a linha

105 testa rejeicao de senha invalida.

(Mostre test_tgs.py linha 94.)

Frase para ler:

Em test_tgs.py, linha 94, existe teste para rejeitar autenticador

reutilizado, cobrindo protecao contra replay no TGS.

(Mostre test_notas.py linhas 170, 203, 278 e 338.)

Frase para ler:

Em test_notas.py, a linha 170 testa autenticacao mutua no Portal. A linha

203 testa replay no Portal. A linha 278 testa edicao e exclusao passando por

operacao Kerberos, e a linha 338 testa bloqueio de aluno ao tentar lancar nota.

(Mostre test_rede.py linhas 96, 191 e 245.)

Frase para ler:

O arquivo test_rede.py e um dos mais importantes. A linha 96 testa o fluxo

completo passando por tres servidores TCP. A linha 191 confirma que a senha nao atravessa a rede. A linha 245 testa a aplicacao web usando os servidores TCP.

(Mostre o comando no terminal.)

```
python -m pytest -q
```

Frase para ler:

O resultado esperado da suite e 48 testes passando.

3. Documentacao Doxygen

(Mostre Doxyfile linhas 5, 6, 15, 18 e 19.)

Frase para ler:

A documentacao Doxygen tambem esta configurada. No Doxyfile, a linha 5 define as entradas, a linha 6 usa o layout customizado, a linha 15 gera HTML, a linha 18 define a pagina principal e a linha 19 aplica o CSS customizado.

(Mostre DoxygenLayout.xml linha 7.)

Frase para ler:

No DoxygenLayout.xml, linha 7, aparece a aba "Como executar o codigo", apontando para como_executar.html.

(Mostre doxygen_pages/execucao.md linha 1.)

Frase para ler:

A pagina execucao.md, linha 1, define o identificador Doxygen como_executar, que gera a pagina de execucao da documentacao.

4. Limitacoes academicas

(Mostre README.md, secao de limitacoes.)

Frase para ler:

Como o projeto e academico, existem simplificacoes. A persistencia usa JSON, as sessoes e caches de replay ficam em memoria, e a execucao e local. Em producao, seria necessario HTTPS, gestao mais forte de segredos e banco de dados.

(Mostre scripts/gerar_chaves.py, se quiser citar as chaves.)

Frase para ler:

As chaves padrao sao didaticas, mas podem ser substituidas por variaveis de ambiente. O script gerar_chaves.py ajuda a gerar novos valores.

5. Conclusao

(Mostre docs/fluxo_kerberos.md ou o README com o fluxo.)

Frase para ler:

Para concluir, o projeto atende aos requisitos minimos da implementacao. Ele usa criptografia simetrica, implementa AS, implementa TGS, protege um sistema de notas, autentica por senha, deriva chave com KDF, emite e valida tickets, implementa autenticacao mutua e segue o fluxo Cliente, AS, TGS e Servico.

(Mostre os terminais com logs ou a tela do Portal.)

Frase para ler:

O ponto mais importante e que o Kerberos nao ficou apenas no login. O fluxo inicial autentica o usuario, e as operacoes do Portal continuam protegidas com Service Ticket, autenticador, nonce, timestamp, hash e resposta cifrada.

(Mostre uma tela final limpa, como README ou o Portal.)

Frase para ler:

Assim, o Portal de Notas funciona como uma demonstracao completa e didatica dos principais conceitos do Kerberos dentro do escopo da disciplina de Seguranca Computacional.

Checklist final para gravacao

- Abrir dois terminais.
- Rodar python scripts/iniciar_servidores.py.
- Rodar python run.py.
- Abrir http://127.0.0.1:5000.
- Nao mostrar senhas.
- Mostrar run.py linha 22.
- Mostrar scripts/iniciar_servidores.py linhas 57 a 66.
- Mostrar kdf.py linhas 29 a 31 e 106 a 111.
- Mostrar as_server.py linhas 128, 276, 330, 353 e 358.
- Mostrar as_server.py linhas 246 a 254 para o TGT.
- Mostrar tgs_server.py linhas 99, 152, 224 e 267.
- Mostrar portal_notas.py linhas 192, 222 a 224 e 401.
- Mostrar routes.py linhas 518, 537, 560 a 570 e 603.

- Mostrar professor lançando ou editando nota.
- Mostrar aluno sem botoes de alteracao.
- Mostrar python -m pytest -q com 48 testes.
- Mostrar Doxygen ou Doxyfile.

Frase curta para encerrar se faltar tempo

O projeto demonstra o Kerberos de ponta a ponta: Cliente, AS, TGS e Portal de Notas. A senha fica no inicio, a KDF deriva a chave, o AS emite o TGT, o TGS emite o Service Ticket, o Portal faz autenticacao mutua e cada operacao de notas continua protegida por ticket, autenticador, nonce, timestamp e hash.